Lec-24: Translation Units, Linkage & ODR

Function Template – Case 1

header.hpp

```cpp
template<typename T>
void f(T x) {};
// DECLARATION + DEFINITION
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    f('a');     // f<char>()
}
```

source.cpp

```cpp
#include "header.hpp"

void h() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    f('a');     // f<char>()
}
```

Predict: compile? link? run correctly?

Function Template – Case 1

header.hpp

```cpp
template<typename T>
void f(T x) {};
// DECLARATION + DEFINITION
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    f('a');     // f<char>()
}
```

source.cpp

```cpp
#include "header.hpp"

void h() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    f('a');     // f<char>()
}
```

Observation:

- Instantiations occur in *both* TUs
- `f<int>`, `f<float>`, `f<char>`
- Identical copies generated; linker merges
- Allowed by template ODR rules

Checking Template Instantiations (GCC/Clang)

Step 1: Compile separately

```
g++ -c main.cpp
g++ -c source.cpp
```

Step 2: Inspect object files

```
nm -C main.o | grep f
nm -C source.o | grep f
```

Output (both files):

```
T void f<int>(int)
T void f<float>(float)
T void f<char>(char)
```

Observation:

- Same instantiations appear in both TUs
- Duplicate symbols generated

Step 3: Link

```
g++ main.o source.o -o app
```

Step 4: Inspect final binary

```
nm -C app | grep f
```

Output:

```
T void f<int>(int)
T void f<float>(float)
T void f<char>(char)
```

Conclusion:

- Linker keeps only one copy
- Duplicate instantiations are merged
- Works only if definitions are identical

Function Template – Case 2

header.hpp

```cpp
template<typename T>
void f(T x);
// DECLARATION only
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    // f('a');  // linking error
}
```

source.cpp

```cpp
#include "header.hpp"

template<typename T>
void f(T x) {}
// DEFINITION

// Explicit instantiations
template void f<int>(int);
template void f<float>(float);
```

Predict: compile? link? run correctly?

Function Template – Case 2

header.hpp

```cpp
template<typename T>
void f(T x);
// DECLARATION only
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    f(3);       // f<int>()
    f(3.3f);    // f<float>()
    // f('a');  // linking error
}
```

Note:

- `f<char>()` is not instantiated here
- If called → **linker error**

source.cpp

```cpp
#include "header.hpp"

template<typename T>
void f(T x) {}
// DEFINITION

// Explicit instantiations
template void f<int>(int);
template void f<float>(float);
```

Observation:

- Only explicitly instantiated specialisations exist
- No implicit instantiation in `main.cpp`
- Missing ones → linker error

Function Template and Specialization

header.hpp

```cpp
template<typename T>
void f() {}
// Generic: DECLARATION + DEFINITION
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    f<char>(); // uses generic
    f<int>();  // uses specialization
}
```

source.cpp

```cpp
#include "header.hpp"

template<>
void f<int>() {}
// SPECIALIZATION definition
```

Predict: compile? link? run correctly?

Function Template and Specialization

header.hpp

```
template<typename T>
void f() {}
// Generic: DECLARATION + DEFINITION
```

main.cpp

```
#include "header.hpp"

int main() {
    f<char>(); // uses generic
    f<int>();  // uses specialization
}
```

source.cpp

```
#include "header.hpp"

template<>
void f<int>() {}
// SPECIALIZATION definition
```

Observation:

- `f<char>()` → generic template
- `f<int>()` → specialized version
- Specialization overrides generic for that type

Function Template – extern Keyword (C++11)

header.hpp

```cpp
template<typename T>
void f() {}
// DECLARATION + DEFINITION
```

main.cpp

```cpp
#include "header.hpp"

extern template void f<int>();
// prevents instantiation in this TU

int main() {
    f<int>(); // uses external instance
}
```

source.cpp

```cpp
#include "header.hpp"

void g() {
    f<int>(); // may trigger instantiation
}

// or explicitly:
template void f<int>();
```

Predict: compile? link? run correctly?

Function Template – extern Keyword (C++11)

header.hpp

```
template<typename T>
void f() {}
// DECLARATION + DEFINITION
```

main.cpp

```
#include "header.hpp"

extern template void f<int>();
// prevents instantiation in this TU

int main() {
    f<int>(); // uses external instance
}
```

Note:

- No code generated for `f<int>()` here
- Expects definition elsewhere

source.cpp

```
#include "header.hpp"

void g() {
    f<int>(); // may trigger instantiation
}

// or explicitly:
template void f<int>();
```

Observation:

- `extern template` suppresses instantiation
- Avoids duplicate codegen across TUs
- One TU must provide the instantiation

ODR – Function Template Common Error

header.hpp

```cpp
template<typename T>
void f(); // DECLARATION

// Problematic:
// template<>
// void f<int>() {}
// multiple definitions if included
```

main.cpp

```cpp
#include "header.hpp"

int main() {}
```

source.cpp

```cpp
#include "header.hpp"

// some code
```

Predict: compile? link? run correctly?

ODR – Function Template Common Error

header.hpp

```cpp
template<typename T>
void f(); // DECLARATION

// Problematic:
// template<>
// void f<int>() {}
// multiple definitions if included
```

main.cpp

```cpp
#include "header.hpp"

int main() {}
```

source.cpp

```cpp
#include "header.hpp"

// some code
```

Observation:

- Full specialization behaves like a normal function
- Defined in header → appears in every TU
- Violates **One Definition Rule** (ODR)
- Linker reports multiple definitions

ODR – Function Template Common Error

header.hpp

```cpp
template<typename T>
void f(); // DECLARATION

// Problematic:
// template<>
// void f<int>() {}
// multiple definitions if included
```

main.cpp

```cpp
#include "header.hpp"

int main() {}
```

source.cpp

```cpp
#include "header.hpp"

// some code
```

Observation:

- Full specialization behaves like a normal function
- Defined in header → appears in every TU
- Violates **One Definition Rule** (ODR)
- Linker reports multiple definitions

Fix: mark it `inline`, *or* move the specialization into a single `.cpp`.

Class Template – Case 1

header.hpp

```
template<typename T>
struct A {
    T x = 3;                // "inline"
definition
    static inline T y = 3; // C++17 inline
static
    void f() {};            // "inline"
definition
};
```

main.cpp

```
#include "header.hpp"

int main() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

source.cpp

```
#include "header.hpp"

int g() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

Predict: compile? link? run correctly?

Class Template – Case 1

header.hpp

```
template<typename T>
struct A {
    T x = 3;                // "inline"
definition
    static inline T y = 3; // C++17 inline
static
    void f() {};            // "inline"
definition
};
```

main.cpp

```
#include "header.hpp"

int main() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

source.cpp

```
#include "header.hpp"

int g() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

Observation:

- All definitions live inside the template
- Implicitly **inline**
- Each TU instantiates independently
- No ODR violation (templates exempt)

Class Template – Case 2

header.hpp

```cpp
template<typename T>
struct A {
    static T x;
    void f(); // DECLARATION
};

#include "header.i.hpp"
```

header.i.hpp

```cpp
template<typename T>
T A<T>::x = 3; // DEFINITION

template<typename T>
void A<T>::f() {} // DEFINITION
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

source.cpp

```cpp
#include "header.hpp"

int g() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

Predict: compile? link? run correctly?

Class Template – Case 2

header.hpp

```cpp
template<typename T>
struct A {
    static T x;
    void f(); // DECLARATION
};

#include "header.i.hpp"
```

header.i.hpp

```cpp
template<typename T>
T A<T>::x = 3; // DEFINITION

template<typename T>
void A<T>::f() {} // DEFINITION
```

Observation:

- Definitions still reach every TU via `.i.hpp`
- Implicit instantiation behaves as in Case 1

main.cpp

```cpp
#include "header.hpp"

int main() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

source.cpp

```cpp
#include "header.hpp"

int g() {
    A<int> a1;   // ok
    A<float> a2; // ok
    A<char> a3;  // ok
}
```

Class Template – Case 3

header.hpp

```cpp
template<typename T>
struct A {
    static T x;
    void f(); // DECLARATION
};
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    A<int> a1;    // ok
    // A<char> a2; // linking error
}
```

source.cpp

```cpp
#include "header.hpp"

template<typename T>
T A<T>::x = 3; // definition

template<typename T>
void A<T>::f() {} // definition

// Explicit instantiation
template class A<int>;
```

Predict: compile? link? run correctly?

Class Template – Case 3

header.hpp

```cpp
template<typename T>
struct A {
    static T x;
    void f(); // DECLARATION
};
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    A<int> a1;   // ok
    // A<char> a2; // linking error
}
```

Note:

- `A<char>` not instantiated anywhere
- Both `f()` and `x` would be missing

source.cpp

```cpp
#include "header.hpp"

template<typename T>
T A<T>::x = 3; // definition

template<typename T>
void A<T>::f() {} // definition

// Explicit instantiation
template class A<int>;
```

Observation:

- Only `A<int>` is generated
- No implicit instantiation elsewhere
- Missing types → linker error

Class Template – extern Keyword (C++11)

header.hpp

```cpp
template<typename T>
struct A {
    T x;
    void f() {}
};
```

main.cpp

```cpp
#include "header.hpp"
extern template class A<int>;
// suppress instantiation in this TU

int main() {
    A<int> a; // uses external instance
}
```

source.cpp

```cpp
#include "header.hpp"

// Explicit instantiation
template class A<int>;

// or any usage that forces it:
// A<int> obj;
```

Predict: compile? link? run correctly?

Class Template – extern Keyword (C++11)

header.hpp

```cpp
template<typename T>
struct A {
    T x;
    void f() {}
};
```

main.cpp

```cpp
#include "header.hpp"
extern template class A<int>;
// suppress instantiation in this TU

int main() {
    A<int> a; // uses external instance
}
```

Note:

- No instantiation of `A<int>` here
- Expects it from another TU

source.cpp

```cpp
#include "header.hpp"

// Explicit instantiation
template class A<int>;

// or any usage that forces it:
// A<int> obj;
```

Observation:

- `extern template` suppresses codegen
- Avoids duplicate instantiations
- One TU must provide the definition

Undefined Behavior – `inline` Function

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

void g();

int main() {
    std::cout << f(); // prints 3
    std::cout << g(); // prints 3 (!!)
}
```

source.cpp

```cpp
// same signature + inline
inline int f() { return 5; }

int g() {
    return f();
}
```

Predict: compile? link? run correctly?

Undefined Behavior – `inline` Function

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

void g();

int main() {
    std::cout << f(); // prints 3
    std::cout << g(); // prints 3 (!!)
}
```

Observation:

- Two different definitions of `f()`
- Both marked `inline` → violates ODR

source.cpp

```cpp
// same signature + inline
inline int f() { return 5; }

int g() {
    return f();
}
```

Behavior:

- Linker picks one version for the program
- With `-O3`, `f()` may be inlined into `g()`
- So `g()` can return 3 *or* 5 unpredictably

Undefined Behavior – `inline` Function

main.cpp

```cpp
#include <iostream>

inline int f() { return 3; }

void g();

int main() {
    std::cout << f(); // prints 3
    std::cout << g(); // prints 3 (!!)
}
```

Observation:

- Two different definitions of `f()`
- Both marked `inline` → violates ODR

source.cpp

```cpp
// same signature + inline
inline int f() { return 5; }

int g() {
    return f();
}
```

Behavior:

- Linker picks one version for the program
- With `-O3`, `f()` may be inlined into `g()`
- So `g()` can return 3 *or* 5 unpredictably

Conclusion / Fix:

- UB – silent, hard to detect in large code
- Use `static` or an anonymous namespace

Undefined Behavior – Member Function

header.hpp

```
#include <iostream>

struct A {
    int f() { return 3; }
};

int g();
```

main.cpp

```
#include "header.hpp"

int main() {
    A a;
    std::cout << a.f(); // prints 3
    std::cout << g();   // prints 3 (!!)
}
```

source.cpp

```
struct A {
    int f() { return 5; }
};

int g() {
    A a;
    return a.f();
}
```

Predict: compile? link? run correctly?

Undefined Behavior – Member Function

header.hpp

```cpp
#include <iostream>

struct A {
    int f() { return 3; }
};

int g();
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    A a;
    std::cout << a.f(); // prints 3
    std::cout << g();   // prints 3 (!!)
}
```

source.cpp

```cpp
struct A {
    int f() { return 5; }
};

int g() {
    A a;
    return a.f();
}
```

Behavior:

- Two definitions of `struct A` across TUs
- Violates **ODR**; linker silently merges
- `g()` may return 3 *or* 5

Undefined Behavior – Member Function

header.hpp

```cpp
#include <iostream>

struct A {
    int f() { return 3; }
};

int g();
```

main.cpp

```cpp
#include "header.hpp"

int main() {
    A a;
    std::cout << a.f(); // prints 3
    std::cout << g();   // prints 3 (!!)
}
```

source.cpp

```cpp
struct A {
    int f() { return 5; }
};

int g() {
    A a;
    return a.f();
}
```

Behavior:

- Two definitions of `struct A` across TUs
- Violates **ODR**; linker silently merges
- `g()` may return 3 *or* 5

Fix:

- One header, one definition, shared everywhere

Undefined Behavior – Function Template

header.hpp

```cpp
template<typename T>
int f() { return 3; }

int g();
```

main.cpp

```cpp
#include <iostream>
#include "header.hpp"

int main() {
    std::cout << f<int>(); // prints 3
    std::cout << g();      // prints 3 (!!)
}
```

source.cpp

```cpp
template<typename T>
int f() { return 5; }

int g() {
    return f<int>();
}
```

Predict: compile? link? run correctly?

Undefined Behavior – Function Template

header.hpp

```cpp
template<typename T>
int f() { return 3; }

int g();
```

main.cpp

```cpp
#include <iostream>
#include "header.hpp"

int main() {
    std::cout << f<int>(); // prints 3
    std::cout << g();      // prints 3 (!!)
}
```

source.cpp

```cpp
template<typename T>
int f() { return 5; }

int g() {
    return f<int>();
}
```

Behavior:

- Two template definitions for the same name
- Instantiations of `f<int>()` differ → ODR
- Linker picks either → `g()` returns 3 or 5

Undefined Behavior – Function Template

header.hpp

```cpp
template<typename T>
int f() { return 3; }

int g();
```

main.cpp

```cpp
#include <iostream>
#include "header.hpp"

int main() {
    std::cout << f<int>(); // prints 3
    std::cout << g();      // prints 3 (!!)
}
```

source.cpp

```cpp
template<typename T>
int f() { return 5; }

int g() {
    return f<int>();
}
```

Behavior:

- Two template definitions for the same name
- Instantiations of `f<int>()` differ → ODR
- Linker picks either → `g()` returns 3 or 5

Fix:

- Keep every template in one header, identical everywhere

Reference

- Federico Busato, *Modern C++ Programming*
  Translation Units I
  `federico-busato.github.io/Modern-CPP-Programming`

Questions?